

Principais *Funcionalidades*

Nas aulas 01 e 02 você viu o básico: Tab, Ctrl+K, o Agent e seus modos, Plan Mode, @, rules. Agora é hora de conhecer o resto da caixa de ferramentas: **Debug Mode**, **Canvases**, **subagentes**, **skills**, **MCP**, **hooks**, **plugins**, **revisão de código**, **agentes na nuvem**, **automações**, **integrações** e o CLI em scripts.

Debug Mode

Canvases

Subagentes · Skills

MCP · Hooks · Plugins

Agent Review · Bugbot

Cloud · Automations · Projects

Slack · GitHub · JetBrains

Formação Claude Code 2026 · Baseado em [cursor.com \(Docs: Agent Overview, Debug Mode, Canvases, Subagents, Skills, MCP, Hooks, Plugins, Agent Review, Bugbot, Cloud Agents, Automations, Projects, Slack, GitHub, JetBrains, Headless CLI\)](https://cursor.com/docs), consultados em outubro de 2026 · Telas: reprodução ilustrativa

A caixa de ferramentas *inteira*

ESCREVER

Tab e edição inline (Ctrl+K): a IA no cursor de texto.

DELEGAR

Agent nos modos Agent, Ask, Plan e **Debug**; `/goal` para objetivos longos.

VER

Canvases: relatórios e painéis interativos ao lado do chat.

DIVIDIR

Subagentes com contexto próprio, em primeiro ou segundo plano.

ENSINAR

Rules, **skills** e comandos `/` para repetir bons fluxos.

CONECTAR

MCP, **hooks** e **plugins** ligam o agente às suas ferramentas.

REVISAR

Agent Review nas mudanças locais; **Bugbot** nos pull requests.

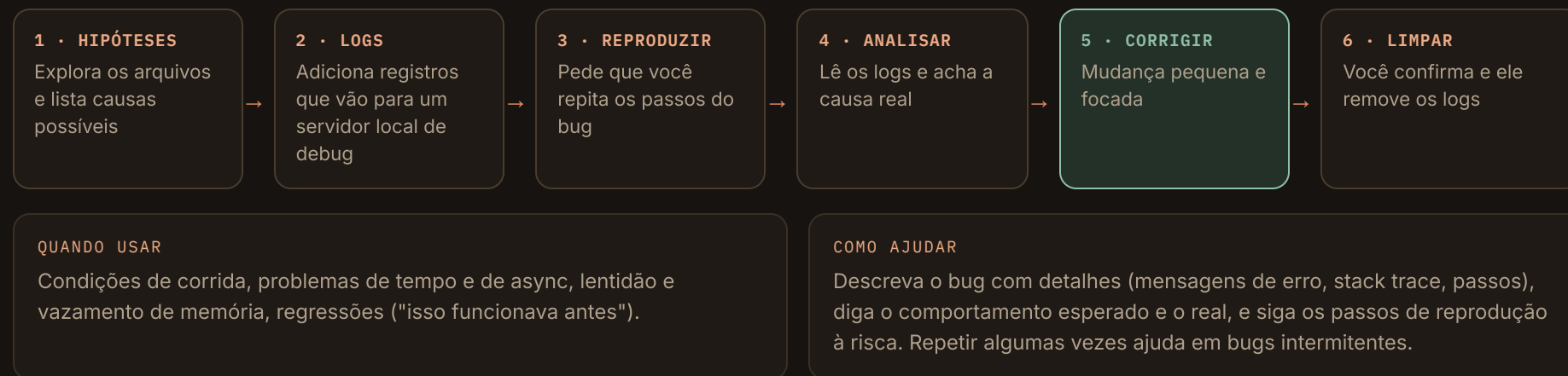
ESCALAR

Cloud Agents, **Automations**, **Projects**, Slack, GitHub e o CLI em scripts.

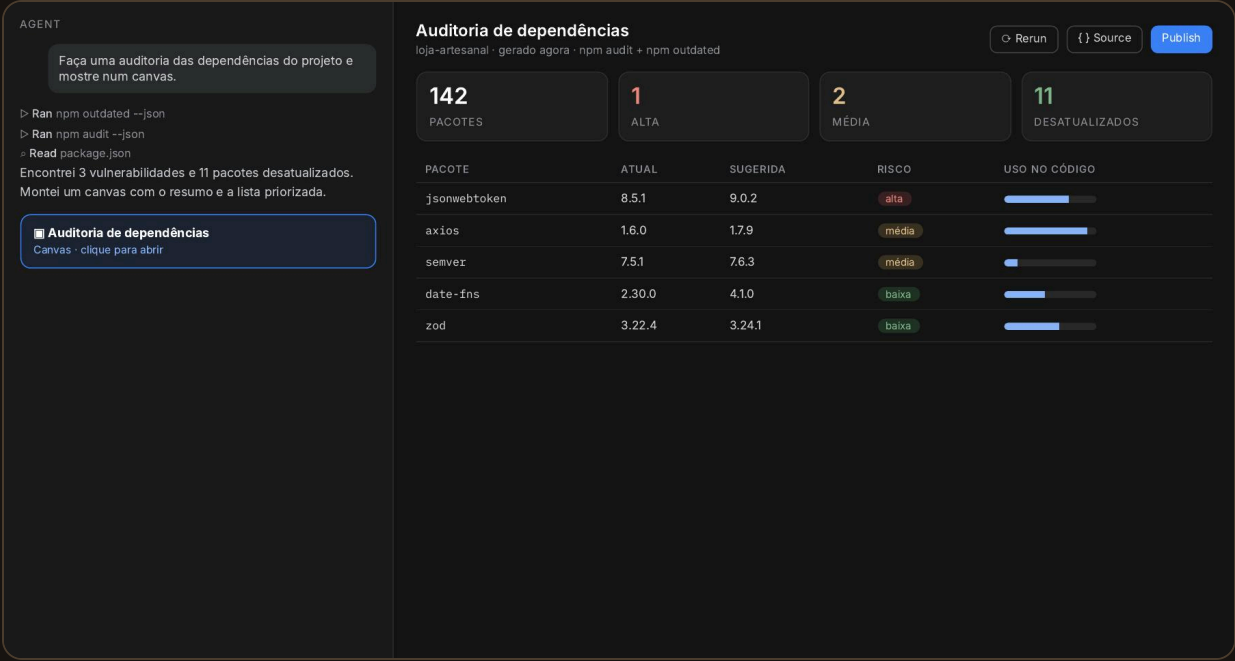
Não é preciso usar tudo. Comece pelo que resolve a sua dor de hoje e acrescente o resto aos poucos. Esta aula mostra **quando** cada recurso vale a pena.

Debug Mode: *evidência em vez de palpite*

Para bugs que você reproduz mas não entende. Em vez de sair escrevendo correções, o agente levanta hipóteses, instrumenta o código e usa dados reais de execução.



Canvases: *respostas que viram painel*



Um pedido de auditoria virou um painel interativo · reprodução ilustrativa

O QUE É

Um artefato interativo (seções, números, tabelas) aberto ao lado do chat. Peça um relatório, uma auditoria ou um painel, ou use `/canvas`.

REUSAR

Fica salvo na lista de canvases do projeto e pode ser **reexecutado** com dados novos. Dá para ajustar pedindo ou editando o código-fonte.

COMPARTILHAR

Publish gera um link para a equipe ver o mesmo painel. Exige plano pago e uma equipe.

Subagentes: *dividir para não lotar o contexto*

Cada subagente tem a **própria janela de contexto** e devolve só o resultado. O Cursor já traz três:

SUBAGENTE	FAZ
Explore	Busca e analisa o código, com um modelo mais rápido e várias buscas em paralelo
Bash	Roda séries de comandos e filtra a saída longa
Browser	Controla o navegador e resume capturas e DOM

Primeiro plano espera o resultado; segundo plano devolve na hora e segue trabalhando sozinho.

```
# .cursor/agents/revisor-seguranca.md
---
name: revisor-seguranca
description: Revisa mudanças em busca de falhas de segurança
model: inherit
readonly: true
---
Você revisa código com foco em segurança: entradas
não validadas, segredos expostos, injeção de SQL...
```

Ficam em `.cursor/agents/` (projeto) ou `~/.cursor/agents/` (você). Por compatibilidade, o Cursor também lê `.claude/agents/`. Ou peça: `/create-subagent`.

Skills: *pacotes de como fazer*

Uma skill é uma pasta com um `SKILL.md` e, se precisar, scripts, modelos e referências. Segue o padrão aberto **Agent Skills**, que funciona em outros agentes. O agente carrega a skill quando ela é relevante; você também chama com `/nome`.

ONDE FICA	ALCANCE
<code>.cursor/skills/</code> · <code>.agents/skills/</code>	Projeto
<code>~/.cursor/skills/</code> · <code>~/.agents/skills/</code>	Seu usuário
<code>.claude/skills/</code> · <code>.codex/skills/</code>	Lidas por compatibilidade

Algumas skills que já vêm no Cursor:

SKILL	PARA QUÊ
<code>/create-rule</code> · <code>/create-skill</code>	Criar rules e skills do jeito certo
<code>/create-hook</code> · <code>/create-subagent</code>	Criar hooks e subagentes
<code>/canvas</code>	Montar um canvas
<code>/automate</code>	Criar uma automação
<code>/autopilot</code>	Cuidar de um PR até ficar pronto
<code>/loop</code>	Repetir um pedido em intervalos
<code>/review</code> · <code>/review-security</code>	Revisão de código e de segurança

Custom Mode: escolha a skill no menu `/` e use `Alt + Enter` (Mac: `Opt + Enter`): ela fica ativa a conversa toda, como um checklist de revisão ou um `/tdd`.

Comandos do chat *que valem decorar*

COMANDO	O QUE FAZ	EXEMPLO
<code>/goal</code>	Dá ao agente um objetivo de longo prazo, perseguido até ser concluído (ainda em liberação gradual)	<code>/goal corrigir os testes instáveis e deixar o CI verde</code>
<code>/side</code> · <code>/btw</code>	Abre uma conversa lateral sem atrapalhar a principal	<code>/btw o que faz esta função?</code>
<code>/in-cloud</code>	Passa a tarefa a um subagente na nuvem (Agents Window)	Tarefas longas
<code>/autopilot</code>	Acompanha um PR: comentários, conflitos e checks que falham	Depois de abrir o PR
<code>/agent-review</code>	Revisão de código das mudanças locais	Antes do commit
<code>/loop</code>	Roda um pedido ou skill de tempos em tempos	Acompanhar um deploy
<code>/create-rule</code>	Cria uma rule com o escopo certo	"sempre use pnpm"

Digite `/` no chat para ver a lista completa, incluindo as skills que você ou a sua equipe criaram.

MCP: *o agente nas suas ferramentas*

O **Model Context Protocol** conecta o agente a sistemas externos: bancos de dados, APIs, Figma, Linear, Notion... Instale pelo **Customize** (marketplace oficial) ou configure à mão.

TRANSPORTE	RODA	USO TÍPICO
stdio	Local, iniciado pelo Cursor	Ferramenta só sua
SSE	Local ou remoto	Servidor compartilhado
Streamable HTTP	Local ou remoto	Servidor compartilhado, com OAuth

```
# .cursor/mcp.json (projeto) ou ~/.cursor/mcp.json (global)
{
  "mcpServers": {
    "meu-servidor": {
      "command": "npx",
      "args": ["-y", "mcp-server"],
      "env": { "API_KEY": "valor" }
    }
  }
}
```

Cada conexão MCP e cada chamada de ferramenta pedem aprovação (ou uma lista de permitidas). Instale só servidores de fonte confiável.

Hooks: *scripts nos eventos do agente*

Hooks rodam scripts seus em momentos do ciclo do agente. Servem para:

- ✓ Formatar o código depois de cada edição
- ✓ Procurar segredos ou dados pessoais
- ✓ Barrar operações arriscadas (ex.: escrita em SQL)
- ✓ Injetar contexto no início da sessão
- ✓ Registrar métricas de uso

```
# .cursor/hooks.json
{
  "version": 1,
  "hooks": {
    "afterFileEdit": [{ "command": "./hooks/format.sh" }]
  }
}
```

ALGUNS EVENTOS	QUANDO
<code>sessionStart</code> / <code>sessionEnd</code>	Início e fim da sessão
<code>beforeSubmitPrompt</code>	Antes de enviar o pedido
<code>preToolUse</code> / <code>postToolUse</code>	Antes e depois de qualquer ferramenta
<code>beforeShellExecution</code>	Antes de um comando no terminal
<code>beforeMCPExecution</code>	Antes de uma ferramenta MCP
<code>beforeReadFile</code> / <code>afterFileEdit</code>	Leitura e edição de arquivos
<code>subagentStart</code> / <code>subagentStop</code>	Ciclo dos subagentes

O Cursor também carrega hooks de ferramentas como o **Claude Code**. Os hooks do projeto rodam até nos agentes da nuvem; os de `~/.cursor`, não.

Plugins *e a página Customize*

Um **plugin** empacota rules, skills, agentes, comandos, servidores MCP e hooks num pacote só, fácil de instalar e de compartilhar.

DOIS FORMATOS

Agent Plugins (padrão aberto: skills e MCP, com `plugin.json`) e **Cursor Plugins** (acrescentam rules, agentes, comandos e hooks, com `.cursor-plugin/plugin.json`).

ONDE ACHAR

No **Cursor Marketplace** (oficiais), em `cursor.directory` (comunidade) ou no marketplace interno da sua equipe (Teams).

ONDE GERENCIAR

Na página **Customize**, na barra lateral: plugins, MCP, rules, skills e subagentes, por usuário, equipe ou projeto.

Plugin é código de terceiros rodando junto com o agente. Antes de instalar, veja quem publica e o que ele inclui, principalmente hooks e servidores MCP.

Agent Review e Bugbot

Envia e-mail quando o pedido é enviado #57

Open feature/email-envio → main · 3 arquivos alterados

```
src/lib/pedidos.ts
export async function atualizarStatus(id: string, status: Status) {
+  const pedido = await db.pedidos.update(id, { status });
+  if (status === "Enviado") enviarEmail(pedido.cliente.email, pedido);
  return pedido;
}
```

cursor bot · Bugbot

agora

Alta

Promise de `enviarEmail` não é aguardada. Se o envio falhar, o erro é perdido e a função retorna como se tudo tivesse dado certo; o cliente não recebe o aviso e nada fica registrado. Use `await` e trate a falha (por exemplo, registrando para reenviar).

Fix in Cursor

Fix in Web

Checks

✓ tests · 48 passed

✓ lint

✗ Cursor Bugbot · 1 issue

Como acionar de novo

Comente `bugbot run` ou `cursor review` no PR. Regras do projeto em `.cursor/BUGBOT.md`.

O Bugbot comenta direto no pull request · reprodução ilustrativa

AGENT REVIEW · ANTES DO COMMIT

Revisa as **mudanças locais**: automático depois de cada commit, com `/agent-review` ou pela aba Source Control (compara com a main). Profundidade **Quick** (rápida, barata) ou **Deep** (lógica complexa, código sensível).

BUGBOT · NO PULL REQUEST

Revisa cada atualização do PR e comenta com explicação e sugestão. Para acionar à mão, comente `bugbot run` ou `cursor review`. **Fix in Cursor** e **Fix in Web** levam direto à correção. Regras do projeto em `.cursor/BUGBOT.md`.

Cloud Agents *na prática*

Agentes em **máquinas virtuais isoladas**, com um ambiente parecido com o seu: repositório clonado, dependências, segredos, comandos de inicialização e rede.

DE ONDE ACIONAR	COMO
Agents Window	Novo agente na nuvem ou <code>/in-cloud</code>
CLI	Comece a mensagem com <code>&</code>
Web e celular	<code>cursor.com/agents</code> e app de iOS
Slack · GitHub · Linear	Mencionando o Cursor

- **Sem repositório:** desde agosto de 2026 dá para começar sem GitHub e salvar o trabalho num repositório do Cursor (Cursor Origin).
- **Vai e volta:** traga o agente para o seu computador para iterar e devolva para ele seguir sozinho.
- **Máquinas próprias:** dá para rodar as ferramentas na sua máquina ou num grupo de máquinas da equipe, com o laço do agente na nuvem.
- **Cobrança:** pelo uso de modelos, como o resto.

O código fica na nuvem enquanto o agente trabalha. Confira a política de privacidade do projeto antes.

Automations: *agentes que acordam sozinhos*

Uma automação roda um agente na nuvem **por agenda** ou **por evento** (GitHub, GitLab, Slack, Linear, webhooks...).

1 Escolha o gatilho: "toda hora", "quando abrirem um PR".

2 Escreva as instruções do agente.

3 Escolha as ferramentas: enviar no Slack, comentar no PR, MCP.

4 Diga se precisa de um, vários ou nenhum repositório.

5 Salve e ative.

```
/automate Toda segunda às 9h, revise os PRs da  
semana passada procurando bugs e mande um resumo  
no canal #engenharia do Slack.
```

JÁ PRONTAS NO PAINEL

Bugbot, **Security Agents** (revisão e varredura de vulnerabilidades) e **PR Routing & Approval** (encaminha PRs e pode aprovar mudanças de baixo risco).

Crie na Agents Window, em cursor.com/automations ou com `/automate`. Cada execução é um agente na nuvem e é cobrada como tal.

Projects: *trabalho de semanas*

COORDENADOR

Você conversa com um agente coordenador. Ele **não escreve código**: planeja, cria e acompanha outros agentes, e traz o resultado para você conferir.

CONTEXTO COMPARTILHADO

Arquivos sincronizados entre todas as máquinas do projeto: pesquisas, artefatos e o que os agentes aprendem sobre o código e o seu jeito de trabalhar.

ASSINATURAS

Peça para ele acompanhar um canal do Slack, seus PRs, o CI ou uma agenda. Ele age sozinho; o selo **Listening** mostra o que está sendo observado.

Bom para: uma funcionalidade com vários PRs, uma migração, um app inteiro, um trabalho para seguir enquanto você está fora.

Roda na nuvem por padrão (abre um agente local quando precisa testar na sua máquina) e **não funciona com o Privacy Mode (Legacy)**.

O Cursor *fora do Cursor*

ONDE	COMO FUNCIONA
Slack	Instale pelo painel de integrações. Mencione <code>@Cursor</code> com o pedido: ele detecta repositório, modelo e branch, e abre um Cloud Agent. <code>@Cursor help</code> lista os comandos
GitHub	O app do Cursor conecta os repositórios (todos ou alguns) e libera Cloud Agents e Bugbot. Também há GitHub Enterprise Server, GitLab, Bitbucket e Azure DevOps
Linear · Jira · Notion · Teams	Acionam agentes a partir de tarefas e conversas
JetBrains	IntelliJ, PyCharm, WebStorm e outras pelo Agent Client Protocol (ACP): no painel AI Chat, <i>Add Agent from Registry</i> → Cursor. Exige plano pago
Xcode	Integração própria para quem desenvolve para Apple
SDK e API	TypeScript e Python, para disparar agentes dos seus sistemas

Na prática: alguém relata um bug no Slack, você responde `@Cursor corrija isso` , e o PR chega pronto para revisão, já com o Bugbot comentando.

O CLI *em scripts e CI*

```
# Modo print (-p): sem interação, ideal para scripts
agent -p "Adicione comentários JSDoc a este arquivo"
# ↑ sem --force, só propõe; não altera arquivos

# Com --force (ou --yolo), aplica as mudanças
agent -p --force "Refatore para a sintaxe ES6+"

# Em lote
find src/ -name "*.js" | while read file; do
  agent -p --force "Adicione JSDoc completo a $file"
done
```

- **CI:** há integração com GitHub Actions para rodar o agente nos seus pipelines.
- **Saída estruturada:** formatos de saída próprios para outros programas lerem.
- **ACP:** `agent acp` transforma o CLI num servidor para outros editores.
- **Mesmas regras:** rules, `mcp.json`, `AGENTS.md` e `CLAUDE.md` valem no CLI.

`--force` deixa o agente alterar arquivos sem confirmação. Use em branch separado, com Git limpo e revisão antes do merge.

Vindo do *Claude Code*

O Cursor lê vários arquivos do ecossistema do Claude Code. Dá para usar as duas ferramentas no mesmo projeto sem duplicar tudo.

NO CLAUDE CODE	NO CURSOR
<code>CLAUDE.md</code>	<code>AGENTS.md</code> e <code>.cursor/rules/*.mdc</code> ; o CLI do Cursor também lê o <code>CLAUDE.md</code>
<code>.claude/skills/</code>	Lidas como estão; o lugar próprio é <code>.cursor/skills/</code> ou <code>.agents/skills/</code>
<code>.claude/agents/</code>	Lidas como subagentes; o lugar próprio é <code>.cursor/agents/</code>
Hooks do Claude Code	Carregados como hooks de terceiros; o formato próprio é <code>.cursor/hooks.json</code>
<code>.mcp.json</code> / servidores MCP	<code>.cursor/mcp.json</code> ou <code>~/.cursor/mcp.json</code> , mesmo protocolo
Plan mode, subagentes, <code>/loop</code>	Plan Mode, subagentes e <code>/loop</code> com a mesma ideia

Skills seguem o padrão aberto **Agent Skills**: a mesma skill pode servir ao Cursor, ao Claude Code e a outros agentes.

Qual recurso *para cada situação*

SITUAÇÃO	RECURSO
Mudança pequena num trecho	Ctrl+K
Funcionalidade em vários arquivos	Plan Mode → Build
Bug que ninguém entende	Debug Mode
Relatório ou auditoria para mostrar à equipe	Canvas
Pesquisa longa que lotaria o contexto	Subagente (ou o Explore, que já vem pronto)
Um jeito de trabalhar que se repete	Skill (ou Custom Mode)
Acessar banco, Figma, Linear...	MCP
Formatar ou barrar algo automaticamente	Hook
Revisar antes do commit / no PR	Agent Review / Bugbot
Tarefa longa com o notebook fechado	Cloud Agent
Algo que precisa rodar toda semana	Automation
Migração ou projeto de semanas	Project

Teste o que *ficou*

1. O que o Debug Mode faz antes de corrigir?

A) Reescreve o arquivo inteiro

B) Levanta hipóteses, adiciona logs e pede que você reproduza o bug

C) Abre um PR

D) Desliga os testes

B. Evidência em tempo de execução.

2. Como acionar o Bugbot manualmente num PR?

A) Comentando `bugbot run` ou `cursor review`

B) Reiniciando o GitHub

C) Só pelo Slack

D) Não é possível

A. E as regras ficam em `.cursor/BUGBOT.md`.

3. Qual a principal vantagem de um subagente?

A) É sempre mais barato

B) Trabalha com o próprio contexto e devolve só o resultado

C) Não precisa de modelo

D) Só roda na nuvem

B. A conversa principal fica limpa.

4. Onde fica a configuração de servidores MCP do projeto?

A) `.cursor/mcp.json`

B) `package.json`

C) `.cursorignore`

D) `AGENTS.md`

A. E `~/.cursor/mcp.json` para todos os projetos.

○ RECAPITULANDO

Uma caixa de ferramentas, *um recurso por problema*

Além de escrever e delegar, o Cursor **depura com evidência** (Debug Mode), **mostra resultados em painéis** (Canvases), **divide o trabalho** (subagentes), **guarda bons fluxos** (skills e comandos `/`), **conecta suas ferramentas** (MCP, hooks, plugins), **revisa** (Agent Review e Bugbot) e **escala** com Cloud Agents, Automations, Projects, integrações e o CLI em scripts. Boa parte é compatível com o que você já usa no Claude Code.

Debug · Canvas

Subagentes · Skills · `/goal`

MCP · Hooks · Plugins

Agent Review · Bugbot

Cloud · Automations · Projects

`agent -p --force`

Próxima aula → Alcançando Maior Produtividade

Fontes: [cursor.com](https://cursor.com/docs) (Docs: Agent Overview, Debug Mode, Canvases, Subagents, Agent Skills, Model Context Protocol, Hooks, Plugins, Agent Review, Bugbot, Cloud Agents, Automations, Projects, Slack, GitHub, JetBrains, Using Headless CLI, Using Agent in CLI, Agent Security), consultados em outubro de 2026. Produto em evolução rápida: nomes de comandos e recursos podem mudar.